Parallel Performance Analysis of π Computation using OpenMP

Vinaykumar Reddy Junuthula
CS 7334.251: HPC@Scale
Texas State University

I. Introduction

Parallel computing plays a critical role in improving the performance of computationally intensive applications. This assignment explores the use of OpenMP to parallelize a numerical integration program that computes the value of π. The goal is to evaluate how different OpenMP runtime parameters such as thread placement, thread count, and scheduling policies influence application performance.

II. System Architecture

Experiments were performed on a compute node of the Lonestar6 cluster at the Texas Advanced Computing Center (TACC). Hardware information was collected using the `hwloc` and `lscpu` utilities.

The compute node specifications are summarized below:

- Processor: AMD EPYC 7763
- Architecture: x86_64
- Total CPUs: 128
- Sockets: 2
- Cores per Socket: 64
- Threads per Core: 1
- NUMA Nodes: 2
- L1 Cache: 32 KB
- L2 Cache: 512 KB
- L3 Cache: 32 MB
- CPU Frequency: $\sim$2.45 GHz

Because the system contains multiple NUMA nodes, thread placement and scheduling strategies can significantly affect performance.

III. OpenMP Parallelization

The sequential program approximates the value of π using numerical integration:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx$$

The computation is divided into N steps and the function value is evaluated at each step.

To parallelize the program, OpenMP directives were used to distribute loop iterations across threads.

A. OpenMP Constructs

The following constructs were used:

- **#pragma omp parallel for** – distributes loop iterations among threads.
- **reduction(+:sum)** – ensures safe accumulation of partial sums.
- **schedule(runtime)** – allows runtime control of scheduling policies.

B. Variable Scope

- Shared variables: `num_steps`, `step`
- Private variables: `i`, `x`
- Reduction variable: `sum`

This approach prevents race conditions while enabling efficient parallel execution. The parallel implementation produced the same value of π as the sequential program up to several decimal places, confirming the correctness of the parallelization.

IV. Experimental Results

All experiments were conducted with:

$$N = 10,000,000$$

Execution time was measured using `omp_get_wtime()`.

A. Thread Placement Policies

The first experiment compared different combinations of OpenMP thread placement parameters:

- core-close
- core-spread
- socket-close
- socket-spread

The results show that **core-spread** achieved the best performance. Spreading threads across cores reduces resource contention and improves overall CPU utilization.

Based on the results in Figure 1, the configuration `OMP_PLACES=core` and `OMP_PROC_BIND=spread` was selected as the best thread placement policy. All subsequent experiments used this configuration.

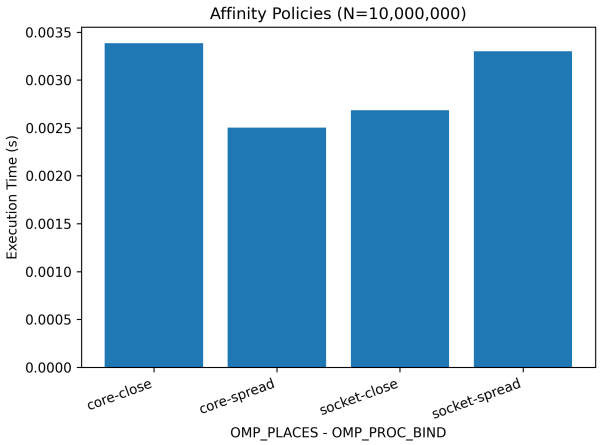

Fig. 1. Execution time for different thread placement policies.

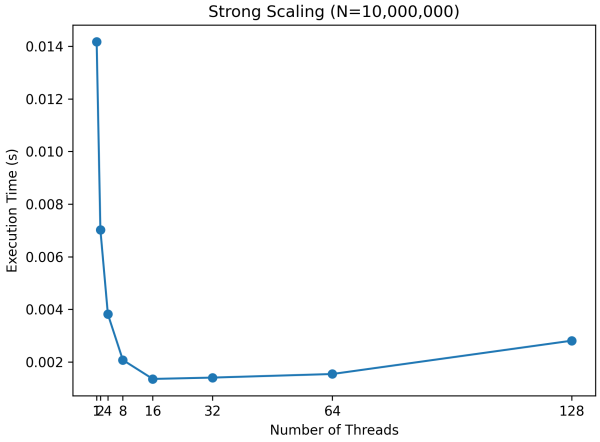

Fig. 2. Execution time vs number of threads.

B. Strong Scaling

The number of threads was varied as:

1, 2, 4, 8, 16, 32, 64, and 128.

Execution time decreases significantly as the number of threads increases up to around 16–32 threads, demonstrating good parallel scalability. Beyond this point, performance improvements diminish due to thread management overhead and memory bandwidth limitations as more cores compete for shared resources.

C. Scheduling Policies

The final experiment compared static and dynamic scheduling with chunk sizes of 10, 100, and 1000.

Dynamic scheduling produced slightly better performance because it allows threads to dynamically acquire new work, improving load balancing across cores. Static scheduling showed slightly higher execution times because work is assigned at the start of execution and cannot be redistributed if load imbalance occurs.

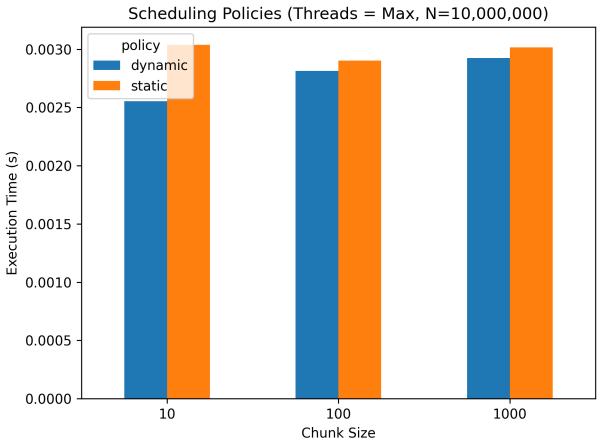

Fig. 3. Performance comparison of static and dynamic scheduling.

V. Conclusion

This assignment demonstrated how OpenMP can be used to parallelize a numerical computation and how runtime parameters influence performance.

The results show that:

- Thread placement significantly impacts performance on NUMA architectures.
- Core-spread placement provided the best performance among the tested configurations.
- The application scales effectively as the number of threads increases up to moderate thread counts.
- Dynamic scheduling slightly improves load balancing compared to static scheduling.

These findings highlight the importance of tuning OpenMP runtime parameters for achieving optimal performance in parallel applications.